

practical-8

```
Problems Output Debug Console Ports
Terminal
PS C:\Users\bhang\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records> Invoke-RestMethod -Uri "http://localhost:8080/auth/register" -Method Post -ContentType "application/json" -Body '{"username":"asha","password":"secret123"}'
message
-----
registered successfully

PS C:\Users\bhang\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records>
```

```
Problems Output Debug Console Ports
Terminal
PS C:\Users\bhang\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records> Invoke-RestMethod -Uri "http://localhost:8080/auth/register" -Method Post -ContentType "application/json" -Body '{"username":"asha","password":"secret123"}'

PS C:\Users\bhang\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records> $token = (Invoke-RestMethod -Uri "http://localhost:8080/auth/login" -Method Post -ContentType "application/json" -Body '{"username":"asha","password":"secret123"}').token
>> $token
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhc2hhIiwiaWF0IjoxNzk3MjYyYXJExLCJleHAiOiJlE3OTA2NjQ0MTF9.SMgTGFd8z8afW7nke1P_7c2CvnnQBQmzs3m9RHrBY0
PS C:\Users\bhang\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records> Invoke-RestMethod -Uri "http://localhost:8080/students" -Method Post -Headers @{ Authorization = "Bearer $token" } -ContentType "application/json" -Body '{"name":"Asha Rao","age":20,"course":"B.Tech CSE","email":"asha@example.com"}'
age : 20
course : B.Tech CSE
email : asha@example.com
id : 6abb5274ec3220c0dac6ebf4
name : Asha Rao

PS C:\Users\bhang\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records>
```

```
Terminal
PS C:\Users\bhang\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records> Invoke-RestMethod -Uri "http://localhost:8080/students" -Method Post -Headers @{ Authorization = "Bearer $token" } -ContentType "application/json" -Body '{"name":"Asha Rao","age":20,"course":"B.Tech CSE","email":"asha@example.com"}'
id : 6abb5274ec3220c0dac6ebf4
name : Asha Rao

PS C:\Users\bhang\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records> Invoke-RestMethod -Uri "http://localhost:8080/students" -Method Get -Headers @{ Authorization = "Bearer $token" }

age : 20
course : B.Tech CSE
email : asha@example.com
id : 6abb5274ec3220c0dac6ebf4
name : Asha Rao

PS C:\Users\bhang\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records>
```

```
Terminal
PS C:\Users\bharg\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records> try {
>>   Invoke-RestMethod -Uri "http://localhost:8080/students" -Method Post -Headers @{ Authorization = "Bearer $token" } -Content
Type "application/json" -Body '{"name":"","age":10,"course":"","email":"not-an-email"}'
>> } catch {
>>   $_.Exception.Response.GetResponseStream() | ForEach-Object { (New-Object System.IO.StreamReader($_)).ReadToEnd() }
>> }
{"details":{"name":"name is required","course":"course is required","email":"a valid email is required","age":"age must be at least 16"},"error":"VALIDATION_ERROR"}
PS C:\Users\bharg\Downloads\TriSem\SEM 4\DBMS\Week 8\student-records>
```

Service

- student_recordss 8.3.7
 - admin
 - config
 - local
 - studentDB
 - test 80K
 - Query
 - Collections (2)
 - students
 - Columns
 - _id Object ID
 - name String
 - age Number
 - course String
 - email String
 - _class String
 - Index
 - users

students X

Properties DATA {} test

db('test').collection('students').find({}).limit(100).toArray()

Search Results

	_id	name	age	course	email	_class
	Object ID	String	Number	String	String	String
	ObjectID('6abb5274ec3220c0dac6ebf4')	Asha Rao	20	B.Tech CSE	asha@example.com	com.example.studentrecord
	_id	ObjectID('6abb5274ec3220c0dac6ebf4')				
	name	Asha Rao				
	age	20				
	course	B.Tech CSE				
	email	asha@example.com				
	_class	com.example.studentrecords.entity.Student				

Java: Ready student_recordss test

Service

- student_recordss 8.3.7
 - admin
 - config
 - local
 - studentDB
 - test 80K
 - Query
 - Collections (2)
 - students
 - Columns
 - _id Object ID
 - name String
 - age Number
 - course String
 - email String
 - _class String
 - Index
 - users
 - Columns
 - _id Object ID
 - username String
 - password String
 - _class String
 - Index

users X

Properties DATA {} test

db('test').collection('users').find({}).limit(100).toArray()

Search Results

	_id	username	password	_class
	Object ID	String	String	String
	ObjectID('6abb5221ec3220c0dac6ebf3')	asha	\$2a\$10\$bncAP0MuGeeC4KjYnUuOsoJQ8Teu5GLZ0KeH4QC6XjKRsQkGG2	com.example.studentrecord
	_id	ObjectID('6abb5221ec3220c0dac6ebf3')		
	username	asha		
	password	\$2a\$10\$bncAP0MuGeeC4KjYnUuOsoJQ8Teu5GLZ0KeH4QC6XjKRsQkGG2		
	_class	com.example.studentrecords.entity.User		

name Nullable

Java: Ready student_recordss test

```
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
32 <dependencies>
55 <dependency>
58 <scope>test</scope>
59 </dependency>
60 <dependency>
61 <groupId>org.springframework.boot</groupId>
62 <artifactId>spring-boot-starter-validation</artifactId>
63 <scope>test</scope>
64 </dependency>
65 <dependency>
66 <groupId>org.springframework.boot</groupId>
67 <artifactId>spring-boot-starter-webmvc</artifactId>
68 <scope>test</scope>
69 </dependency>
70 <dependency>
71 <groupId>io.jsonwebtoken</groupId>
72 <artifactId>jjwt-api</artifactId>
73 <version>0.11.5</version>
74 </dependency>
75 <dependency>
76 <groupId>io.jsonwebtoken</groupId>
77 <artifactId>jjwt-impl</artifactId>
78 <version>0.11.5</version>
79 <scope>runtime</scope>
80 </dependency>
81 <dependency>
82 <groupId>io.jsonwebtoken</groupId>
83 <artifactId>jjwt-jackson</artifactId>
84 <version>0.11.5</version>
85 <scope>runtime</scope>
86 </dependency>
```

```
src > main > resources > application.properties
1 spring.application.name=student-records
2 spring.data.mongodb.uri=mongodb://localhost:27017/student_records
3 jwt.secret=replace-this-with-a-long-random-secret-at-least-32-characters
4 jwt.expiration-ms=3600000
5 server.port=8080
```

```
src > main > java > com > example > studentrecords > entity > User.java
1 package com.example.studentrecords.entity;
2
3 import org.springframework.data.annotation.Id;
4 import org.springframework.data.mongodb.core.index.Indexed;
5 import org.springframework.data.mongodb.core.mapping.Document;
6
7 @Document(collection="users")
8 public class User {
9     @Id
10     private String id;
11     @Indexed(unique=true)
12     private String username;
13     private String password;
14
15     public String getId(){return id;}
16     public void setId(String id){this.id=id;}
17     public String getUsername(){return username;}
18     public void setUsername(String username){this.username=username;}
19     public String getPassword(){return password;}
20     public void setPassword(String password){this.password=password;}
21 }
22
```

The screenshot shows an IDE with the Explorer on the left and the code editor on the right. The Explorer shows a project structure with a package named 'com.example.studentrecords.entity'. The file 'Student.java' is selected. The code editor displays the following code:

```
src > main > java > com > example > studentrecords > entity > J Student.java > {} com.example.studentrecords.entity
1 package com.example.studentrecords.entity;
2
3 import org.springframework.data.annotation.Id;
4 import org.springframework.data.mongodb.core.mapping.Document;
5
6 @Document(collection="students")
7 public class Student {
8     @Id
9     private String id;
10    private String name;
11    private Integer age;
12    private String course;
13    private String email;
14
15    public String getId(){return id;}
16    public void setId(String id){this.id=id;}
17    public String getName(){return name;}
18    public void setName(String name){this.name=name;}
19    public Integer getAge(){return age;}
20    public void setAge(Integer age){this.age=age;}
21    public String getCourse(){return course;}
22    public void setCourse(String course){this.course=course;}
23    public String getEmail(){return email;}
24    public void setEmail(String email){this.email=email;}
25 }
26
```

The screenshot shows an IDE with the Explorer on the left and the code editor on the right. The Explorer shows a project structure with a package named 'com.example.studentrecords.dto'. The file 'LoginRequest.java' is selected. The code editor displays the following code:

```
src > main > java > com > example > studentrecords > dto > J LoginRequest.java > {} com.example.studentrecords.dto
1 package com.example.studentrecords.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4
5 public class LoginRequest {
6     @NotBlank
7     private String username;
8     @NotBlank
9     private String password;
10
11    public String getUsername(){return username;}
12    public void setUsername(String username){this.username=username;}
13    public String getPassword(){return password;}
14    public void setPassword(String password){this.password=password;}
15 }
16
```

The screenshot shows an IDE with the Explorer on the left and the code editor on the right. The Explorer shows a project structure with a package named 'com.example.studentrecords.repository'. The file 'StudentRepository.java' is selected. The code editor displays the following code:

```
src > main > java > com > example > studentrecords > repository > J StudentRepository.java > {} com.example.studentrecords.repository
1 package com.example.studentrecords.repository;
2
3 import com.example.studentrecords.entity.Student;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 public interface StudentRepository extends MongoRepository<Student, String> {}
7
```

```
src > main > java > com > example > studentrecords > service > J StudentService.java > {} com.example.studentrecords.service
1 package com.example.studentrecords.service;
2
3 import com.example.studentrecords.dto.StudentRequest;
4 import com.example.studentrecords.entity.Student;
5 import com.example.studentrecords.repository.StudentRepository;
6 import org.springframework.stereotype.Service;
7 import java.util.*;
8
9 @Service
10 public class StudentService {
11     private final StudentRepository repository;
12
13     public StudentService(StudentRepository repository) {
14         this.repository = repository;
15     }
16
17     public Student create(StudentRequest r) {
18         Student s = new Student();
19         apply(s, r);
20         return repository.save(s);
21     }
22
23     public List<Student> getAll() {
24         return repository.findAll();
25     }
26
27     public Optional<Student> getById(String id) {
28         return repository.findById(id);
29     }
30
31     public Optional<Student> update(String id, StudentRequest r) {
```

```
src > main > java > com > example > studentrecords > exception > J GlobalExceptionHandler.java > {} com.example.studentrecords.exception
1 package com.example.studentrecords.exception;
2
3 import org.springframework.http.ResponseEntity;
4 import org.springframework.web.bind.MethodArgumentNotValidException;
5 import org.springframework.web.bind.annotation.*;
6 import java.util.*;
7
8 @RestControllerAdvice
9 public class GlobalExceptionHandler {
10     @ExceptionHandler(MethodArgumentNotValidException.class)
11     public ResponseEntity<Map<String, Object>>
12         handleValidation(MethodArgumentNotValidException ex) {
13         Map<String, String> errors = new HashMap<>();
14         ex.getBindingResult().getFieldErrors()
15             .forEach(e -> {
16                 errors.put(e.getField(), e.getDefaultMessage());
17             });
18         Map<String, Object> body = new HashMap<>();
19         body.put(key: "error", value: "VALIDATION_ERROR");
20         body.put(key: "details", errors);
21         return ResponseEntity.badRequest().body(body);
22     }
23 }
```

```
src > main > java > com > example > studentrecords > security > J CustomUserDetailsService.java > {} com.example.studentrecords.security
1 package com.example.studentrecords.security;
2
3 import com.example.studentrecords.repository.UserRepository;
4 import org.springframework.security.core.userdetails.*;
5 import org.springframework.stereotype.Service;
6
7 @Service
8 public class CustomUserDetailsService implements UserDetailsService {
9     private final UserRepository userRepository;
10
11     public CustomUserDetailsService(UserRepository userRepository) {
12         this.userRepository = userRepository;
13     }
14
15     @Override
16     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
17         var u = userRepository.findByUsername(username)
18             .orElseThrow(() -> new UsernameNotFoundException("User not found"));
19         return User.withUsername(u.getUsername())
20             .password(u.getPassword())
21             .authorities("USER")
22             .build();
23     }
24 }
25
```

```
src > main > java > com > example > studentrecords > controller > J AuthController.java > {} com.example.studentrecords.controller
1 package com.example.studentrecords.controller;
2
3 import com.example.studentrecords.dto.*;
4 import com.example.studentrecords.entity.User;
5 import com.example.studentrecords.repository.UserRepository;
6 import com.example.studentrecords.security.JwtUtil;
7 import jakarta.validation.Valid;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.authentication.*;
10 import org.springframework.security.crypto.password.PasswordEncoder;
11 import org.springframework.web.bind.annotation.*;
12 import java.util.Map;
13
14 @RestController
15 @RequestMapping("/auth")
16 public class AuthController {
17     private final UserRepository u;
18     private final PasswordEncoder encoder;
19     private final AuthenticationManager manager;
20     private final JwtUtil jwt;
21
22     public AuthController(
23         UserRepository users,
24         PasswordEncoder encoder,
25         AuthenticationManager manager,
26         JwtUtil jwt) {
27         this.users = users;
28         this.encoder = encoder;
29         this.manager = manager;
30         this.jwt = jwt;
31     }
32 }
```

```
PS C:\Users\lhang\Downloads\TriSem\SEM 8\DBMS\Week 8\student-records> .\mvnw.cmd spring-boot:run
monitoring: true, server.tomcat.threads=1, server.tomcat.connections=1, ssl.enabled=false, invalidation.enabled=false, context=null, applicationName=null, compressorList=[], uuidRepresentation=UNSPECIFIED, serverApi=null, autoEncryptionSetting=null, dnsClient=null, inetAddressResolver=null, contextProvider=null, timeoutMS=null)
2026-09-29T11:21:38.011405:30 INFO 8404 --- [student-records] [localhost:27017] org.mongodb.driver.cluster : Monitor thread successfully connected to server with description ServerDescription(address=localhost:27017, type=STANDALONE, cryptd=false, state=CONNECTED, ok=true, minWireVersion=0, maxWireVersion=28, maxDocumentSize=16777216, logicalSessionTimeoutMinutes=30, roundTripTimeNanos=60130900, minRoundTripTimeNanos=0)
2026-09-29T11:21:38.392405:30 INFO 8404 --- [student-records] [main] r$InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsServiceImpl bean with name customUserDetailsService
2026-09-29T11:21:38.88405:30 INFO 8404 --- [student-records] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-09-29T11:21:38.893405:30 INFO 8404 --- [student-records] [main] c.e.s.StudentRecordsApplication : Start
2026-09-29T11:22:33.628405:30 INFO 8404 --- [student-records] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-09-29T11:22:33.629405:30 INFO 8404 --- [student-records] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-09-29T11:22:33.636405:30 INFO 8404 --- [student-records] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
```